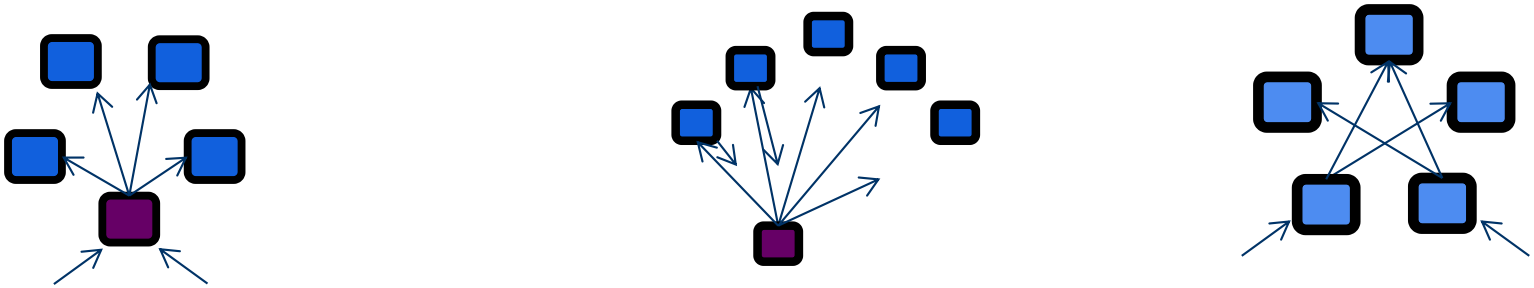
CompSci 390-03-26S

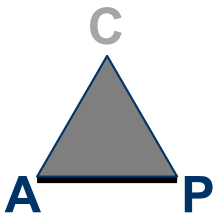
Distributed and Networked Systems

Consistency: Playing Fast and Loose



The story so far: RWN replication

- Choose replica to serve as **coordinator** for read or write.
- Coordinator **multicasts** op to **all** replicas for shard (**N** in total).
- Coordinator waits for a quorum of replies before responding to client.
  - For a read: **R** replies.
  - For a write: **W** replies.
- Configure R and W. Lower R and/or lower W yield:
  - Lower latency (mean response time).
  - Much lower tail latency.
  - Better availability in failure scenarios or network hiccups.
- But what if R and/or W are "too low"? Sacrifice consistency→**unsafe**.



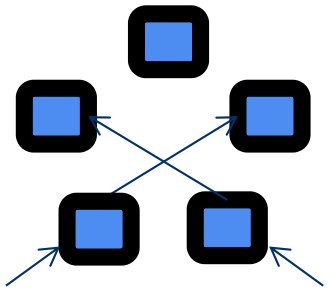
RWN replication: fast and dangerous

For N replicas, we can reduce the read quorum (R) for faster reads and/or the write quorum (W) for faster writes.

But. If we commit reads and writes "too fast", two problems arise:

- **Stale reads** when R + W ≤ N. Client A writes key K, and client B reads K from replica(s) that did not (yet) hear the write. B's read returns stale data.
- **Conflicting** writes when W < [N/2] + 1 (less than a majority). Suppose two clients **concurrently** write key K through different coordinators, and both writes succeed with disjoint write quorums. Replicas diverge on K! The system must **reconcile** the conflict.
- Use **version numbers** to detect/repair.

N=5; R=W=2



RWN replication and tunable consistency

- RWN replication: Dynamo KV store and its successors.
- R,W,N may be tunable per shard or logical "bucket".
- Obtain the "right" tradeoff for your application and network.
- How to quantify staleness?
  - PBS/WARS model
- How to detect/repair write conflicts?
  - Use **version timestamps** to detect
  - Repair version forks: **reconciliation**
  - Application-specific! (Just like in git)

Quantifying Eventual Consistency with PBS

By Peter Bailis, Shivaram Venkataraman, Michael J. Franklin, Joseph M. Hellerstein, and Ion Stoica

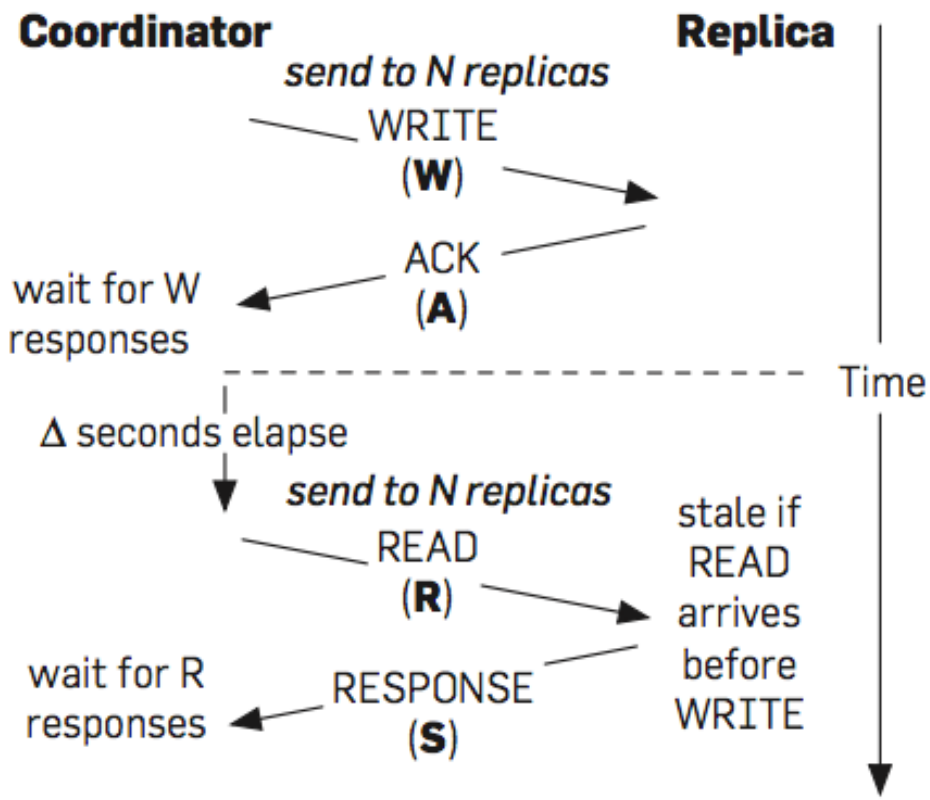
Abstract

Data replication results in a fundamental trade-off between operation latency and consistency. At the weak end of the spectrum of possible consistency models is eventual consistency, which provides no limit to the staleness of data returned. However, anecdotally, eventual consistency is often "good enough" for practitioners given its latency and availability benefits. In this work, we explain this phenomenon and demonstrate that, despite their weak guarantees, eventually consistent systems regularly return consistent data while providing lower latency than their strongly consistent counterparts. To quantify the behavior of eventually consistent stores, we introduce **Probabilistically Bounded Staleness** (PBS), a consistency model that provides expected bounds on data staleness with respect to both versions and wall clock time….

AUGUST 2014 | VOL. 57 | NO. 8 | **COMMUNICATIONS OF THE ACM**

Probabilistically Bounded Staleness: WARS

Figure 2. The *WARS* model describes staleness in Dynamo by modeling message latencies between a coordinator and replicas for a write operation followed by a read operation *t* seconds later. In an *N* replica system, the depicted messages are exchanged with *N* replicas.



t-visibility: what is the probability that a read starting t seconds after a write completes will see that write? **It depends**.

Quantifying Eventual Consistency with PBS

By Peter Bailis, Shivaram Venkataraman, Michael J. Franklin, Joseph M. Hellerstein, and Ion Stoica

Dynamo: multi-version KV store

- Each value has a **version stamp**, also called **context**.
- Why does **Put** take a stamp for the last version read?
- Why can **Get** return multiple values?
- Under what circumstances can this occur?
- If there are multiple values, the app is responsible for reconciling them. **How**?

```
Get(key) → {[version, value]*}
Put(key, version/context, value) → success
```

Dynamo's "killer app": shopping carts

For a number of Amazon services, rejecting customer updates could result in a poor customer experience. For instance, the **shopping cart** service must allow customers to add and remove items from their shopping cart even amidst network and server failures. This requirement forces us to push the complexity of conflict resolution to the reads in order to ensure that writes are never rejected.

Quote from **Dynamo: Amazon's Highly Available Key-value Store**, SOSP 2007

Shopping carts are sets: just take the union

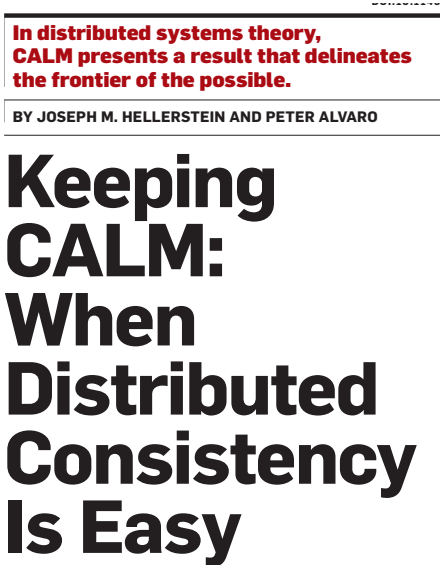
When…the latest version is not available, the item is added to (or removed from) the older version and the divergent versions are reconciled later. …

For example, the application that maintains customer shopping carts can choose to "merge" the conflicting versions and return a single unified shopping cart…Using this reconciliation mechanism, an "add to cart" operation is never lost. **However, deleted items can resurface**.

Quote from **Dynamo: Amazon's Highly Available Key-value Store**, SOSP 2007

CALM and CRDTs

- A shopping cart is a **set**, an example of a **conflict-free replicated data type** (CRDT) with a lattice of values.
- Set union is **commutative, associative, idempotent**: it doesn't matter what order we add items: same output. Reconcile by merging with union.
- Union is a **monotonic** operator: a program P is monotonic if for any input sets S,T where $S \subseteq T$, $P(S) \subseteq P(T)$.
  - "Never take it back."
- But what about deleting items? …

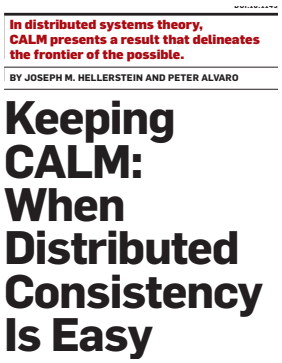

In distributed systems theory, CALM presents a result that delineates the frontier of the possible.

BY JOSEPH M. HELLERSTEIN AND PETER ALVARO

Keeping CALM: When Distributed Consistency Is Easy

Keeping CALM: Sometimes AP gets C for free

- **Theorem**. Consistency As Logical Monotonicity (CALM). A program has a consistent, coordination-free distributed implementation if and only if it is monotonic.
- **Observation**. Coordination-freeness is equivalent to availability under partition.
- CAP is a negative result over *arbitrary* programs.
- CALM is a positive result: "CALM circumscribes the class of problems for which all three of the CAP properties can indeed be achieved simultaneously."


In distributed systems theory, CALM presents a result that delineates the frontier of the possible.

BY JOSEPH M. HELLERSTEIN AND PETER ALVARO

Keeping CALM: When Distributed Consistency Is Easy

COMMUNICATIONS OF THE ACM | SEPTEMBER 2020 | VOL. 63 | NO. 9

Healing partitions: Keep CALM and carry on

- Timeline forks due to partition also result in conflicting writes.
- Partition recovery ➔ Just reconcile all the conflicting writes.
- Partition recovery is easy for CALM applications.

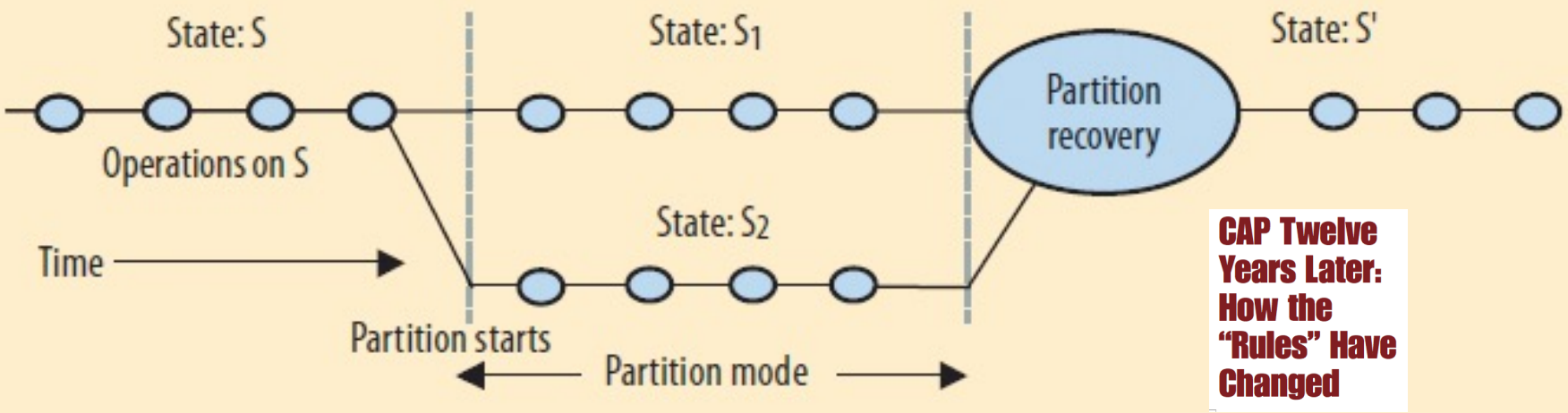


Image borrowed from Eric Brewer *CAP 12 Years Later (2012)*

But many/most applications are not CALM!

- Stay CALM if you can, but most apps need C but lack LM.
- What about them? What can go wrong? What now?
- Preview:
  - "Don't Settle for Eventual": stale reads can scramble apps.
  - Use Lamport's notion of **causality** to capture why.
  - Extend relaxed consistency systems to preserve causality.
  - Causal consistency is the strongest model obtainable in AP systems.
  - These systems must provide a deterministic rollback/reorder/merge to reconcile conflicts and forks with so that all replicas converge.
  - **Logical clocks** and **vector clocks** capture causality and concurrent writes.

Don't Settle for Eventual: Scalable Causal Consistency for Wide-Area Storage with COPS

Wyatt Lloyd*, Michael J. Freedman*, Michael Kaminsky†, and David G. Andersen‡
*Princeton University, †Intel Labs, ‡Carnegie Mellon University

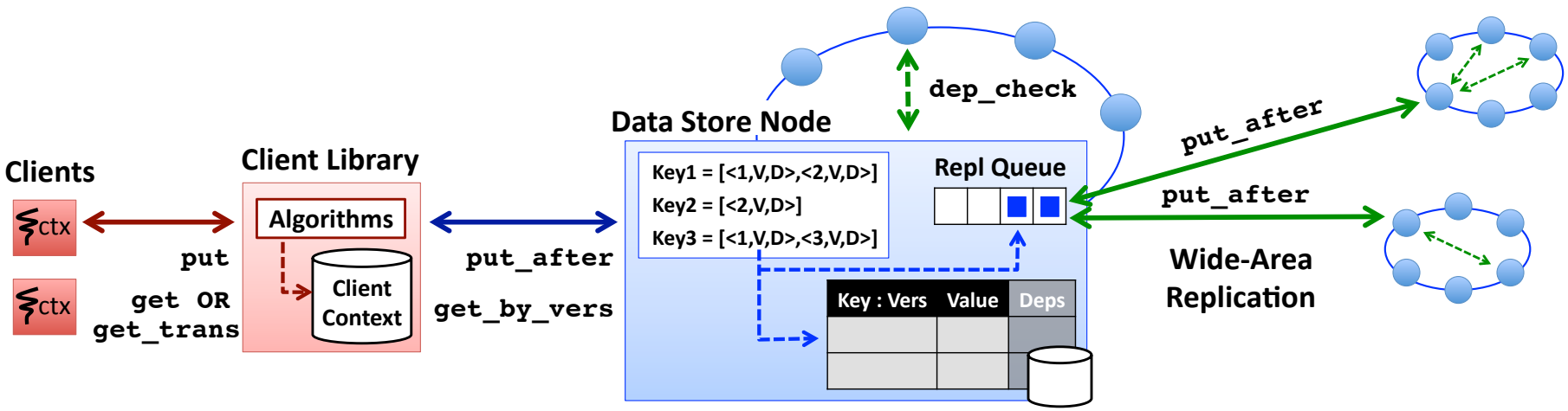
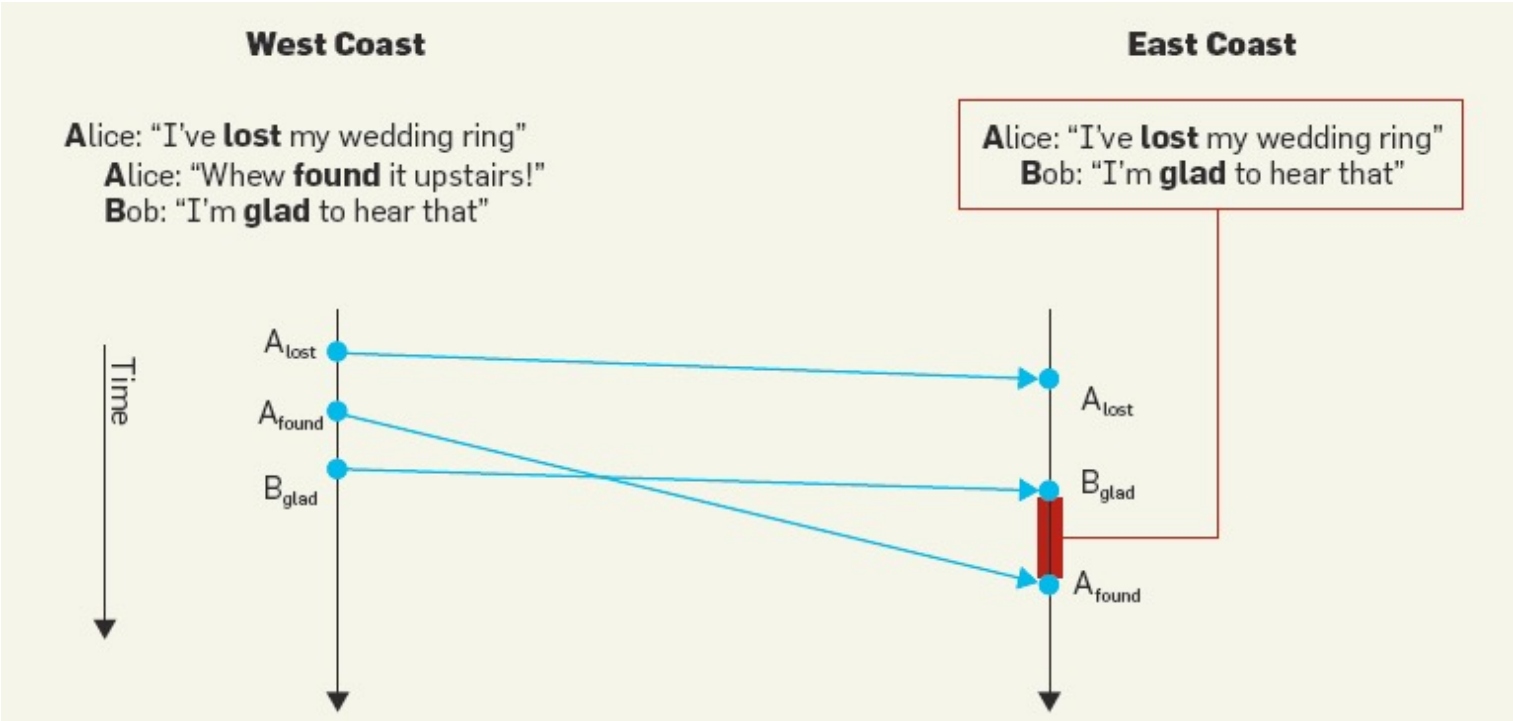


Figure 4: The COPS architecture. A client library exposes a put/get interface to its clients and ensures operations are properly labeled with causal dependencies. A key-value store replicates data between clusters, ensures writes are committed in their local cluster only after their dependencies have been satisfied, and in COPS-GT, stores multiple versions of each key along with dependency metadata.

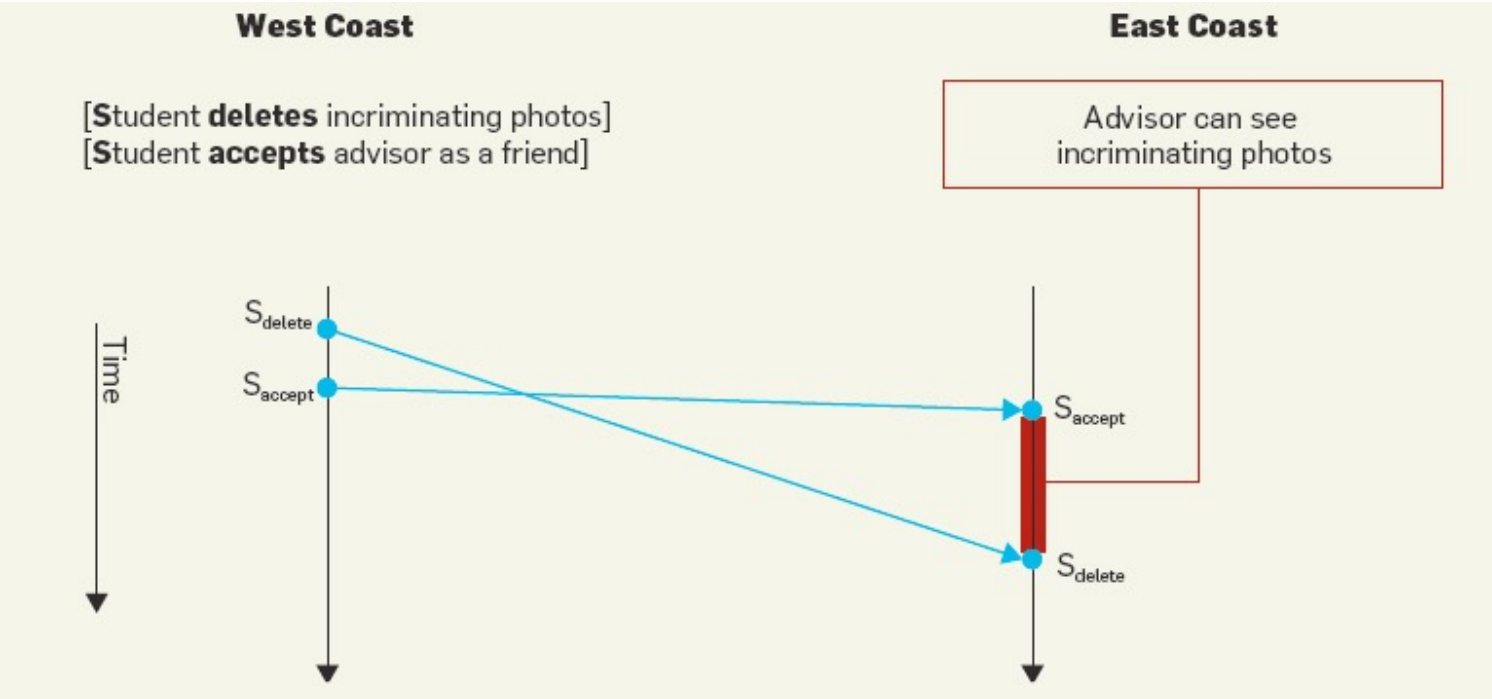
[COPS 2011]

Causal inconsistency: example



[COPS]

Causal inconsistency: example



[COPS]

Multithreaded programs

Initially A = B = 0

Thread 1

```
A = 1
if (B == 0)
    print "Hello";
```

Thread 2

```
B = 1
if (A == 0)
    print "World";
```

What can be printed?

- "Hello"?
- "World"?
- Nothing?
- "Hello World"?

https://www.cs.utexas.edu/~bornholt/post/memory-models.html

Things that shouldn't happen

This program should never print "Hello World".

Thread 1	**Thread 2**
`A = 1`	`B = 1`
`if (B == 0)`	`if (A == 0)`
`  print "Hello";`	`  print "World";`

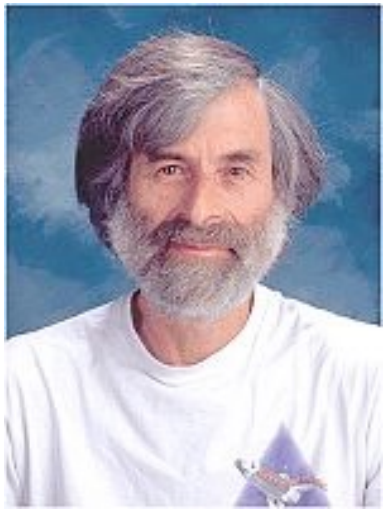
A "happens-before" graph shows the order in which events must execute to get a desired outcome.

- If there's a cycle in the graph, an outcome is impossible—an event must happen before itself!

https://www.cs.utexas.edu/~bornholt/post/memory-models.html

A Classic Paper

- **Time, Clocks, and the Ordering of Events in Distributed Systems**
- Leslie Lamport, CACM, July 1978
- Introduced:
  - logical clocks ("Lamport clocks")
  - state-machine replication model
  - causal multicast / propagation
  - physical clocks respecting causality
  - (Almost but not) vector clocks



“Space-time diagram”

Fig. 1.

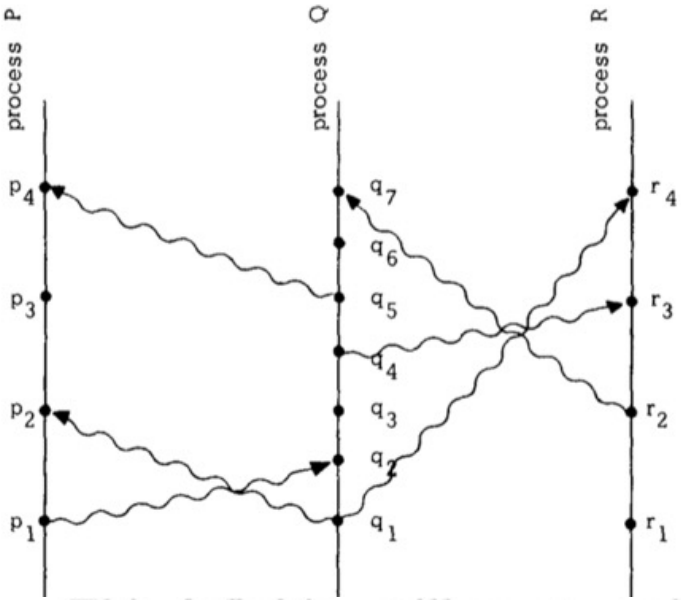
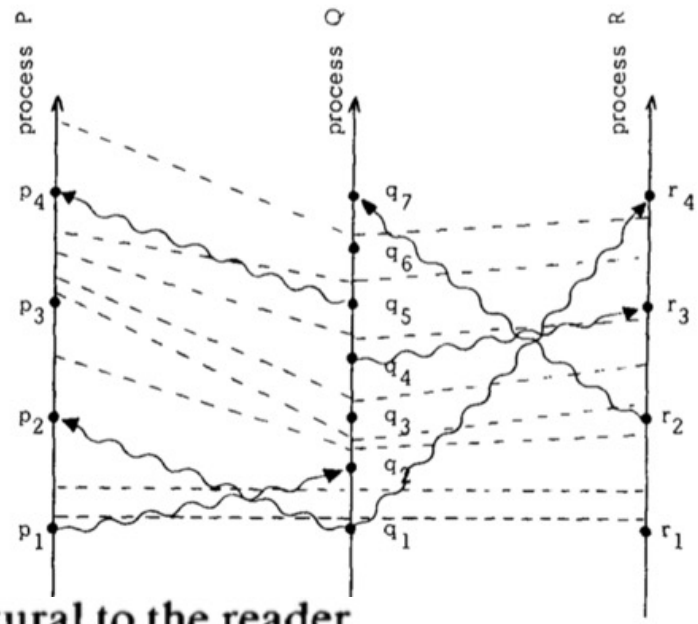
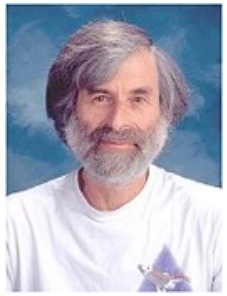


Fig. 2.





This definition will appear quite natural to the reader familiar with the invariant space-time formulation of special relativity, as described for example in [1] or the first chapter of [2]. In relativity, the ordering of events is defined in terms of messages that *could* be sent. However,

DOI:10.1145/2890782

Article development led by acmqueue
queue.acm.org

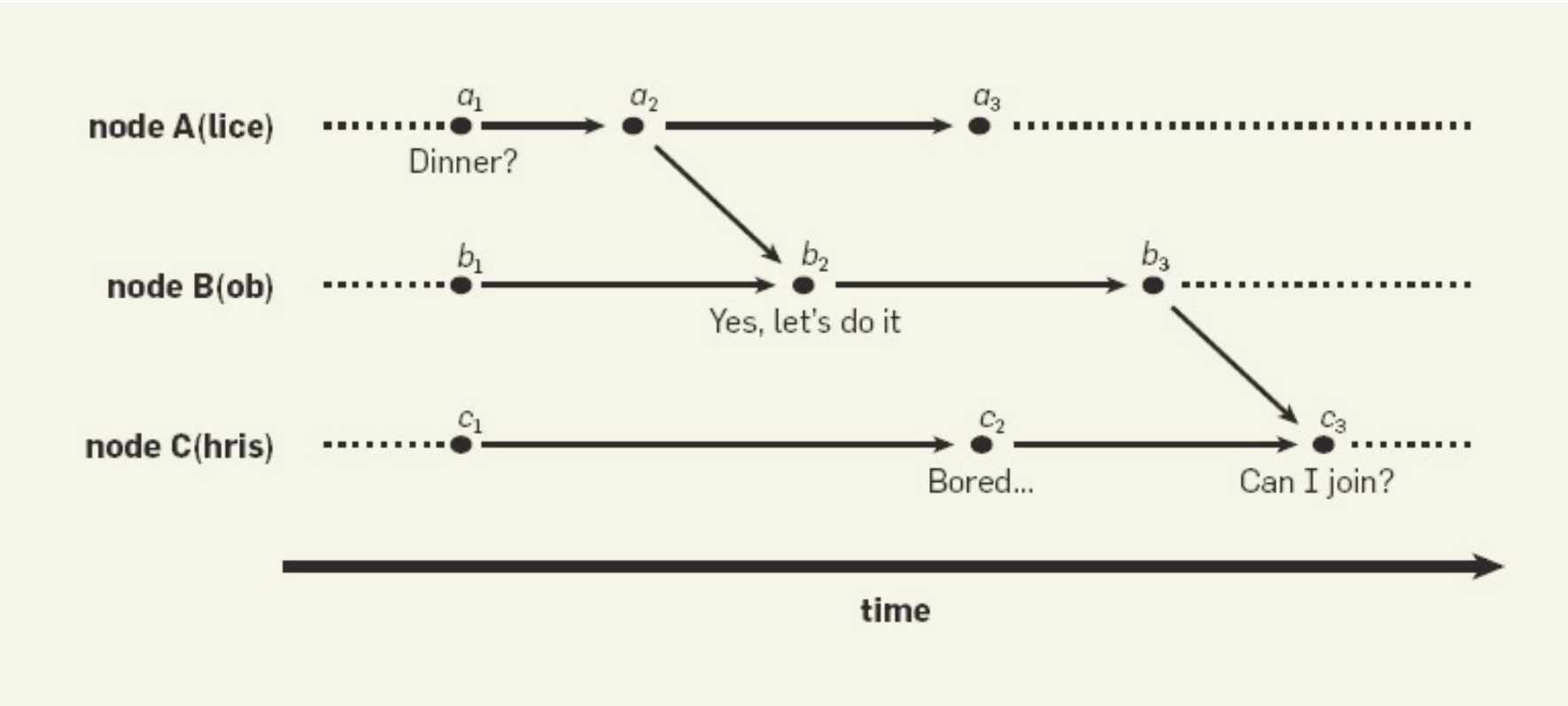
Sometimes all you need is the right language.

BY CARLOS BAQUERO AND NUNO PREGUIÇA

Why Logical Clocks Are Easy

APRIL 2016 VOL. 59 NO. 4
COMMUNICATIONS OF THE ACM

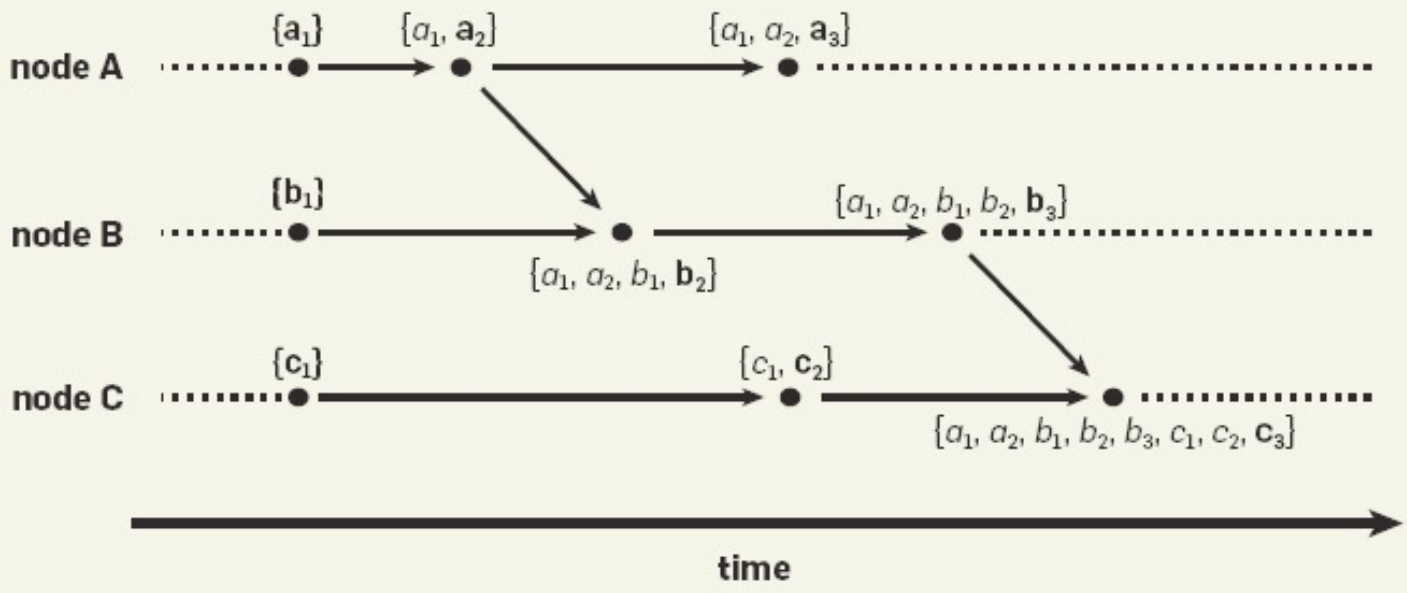
Happened-before



APRIL 2016 VOL. 59 NO. 4 COMMUNICATIONS OF THE ACM

Causal dependencies

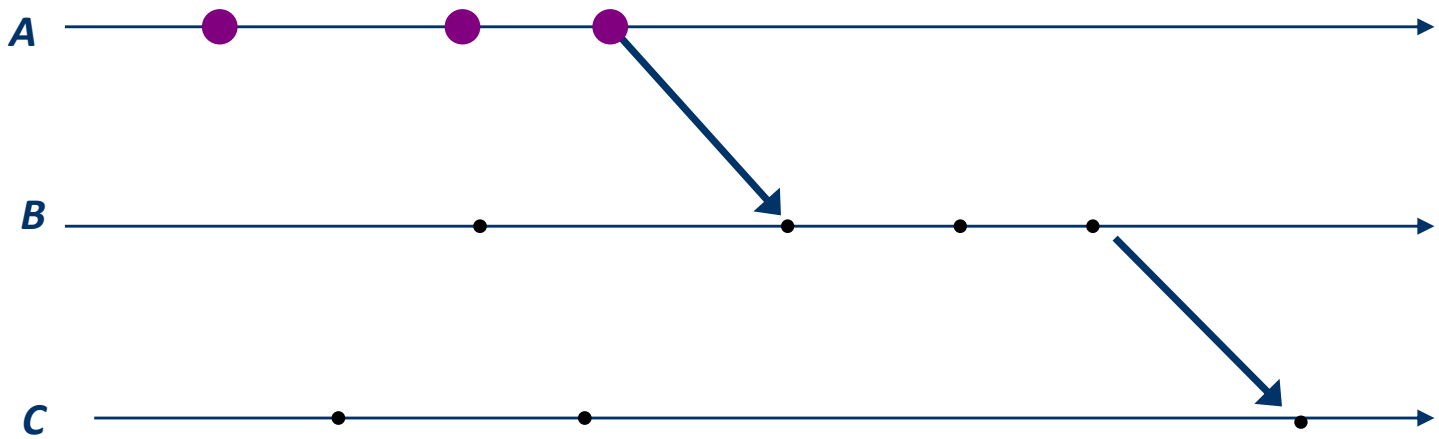
Lists the events that **causally precede** each timeline event:

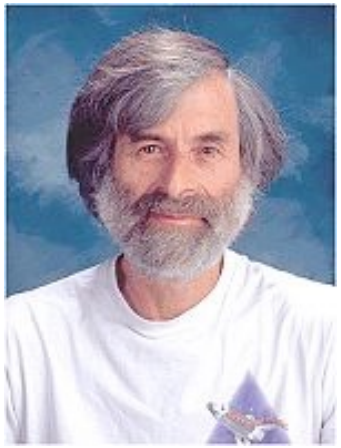


APRIL 2016 VOL. 59 NO. 4 COMMUNICATIONS OF THE ACM

Axiom 1: happened-before (→)

- If *e1, e2* are in the same process/thread, and *e1* comes before *e2*, then *e1*→*e2*.
  - Also called **program order**

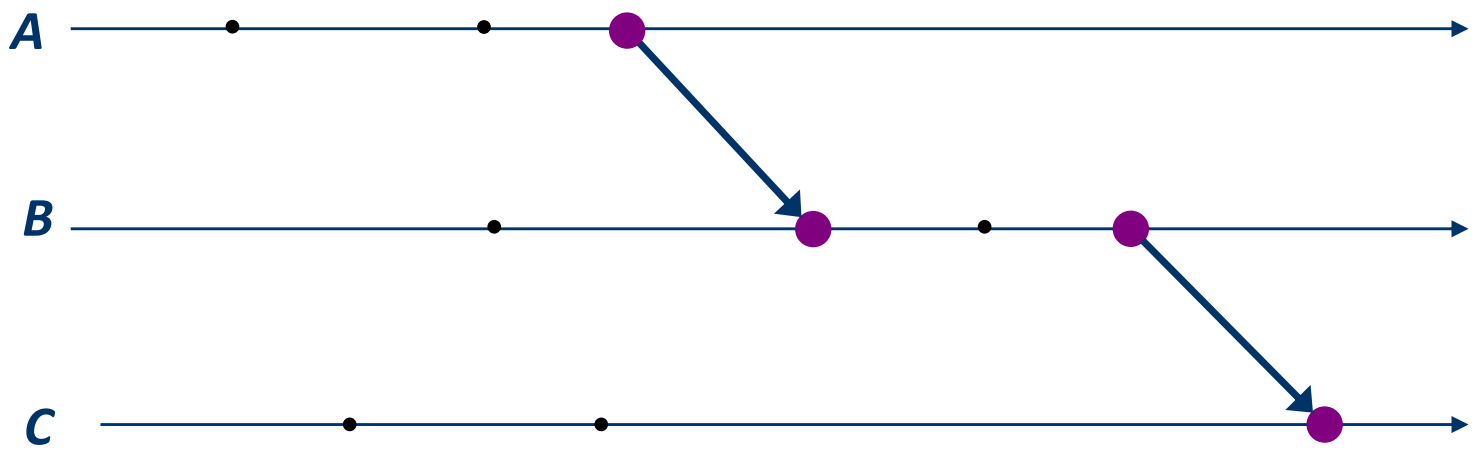


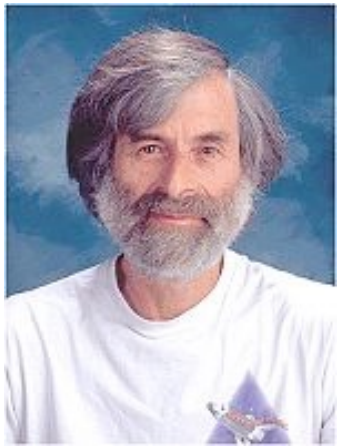


Time, Clocks, and the Ordering of Events in Distributed systems, by Leslie Lamport, CACM 21(7), July 1978

Axiom 2: happened-before (→)

- If e1 is a message send, and e2 is the corresponding receive, then e1→e2. [Also: mutex handoff: release→next acquire.]

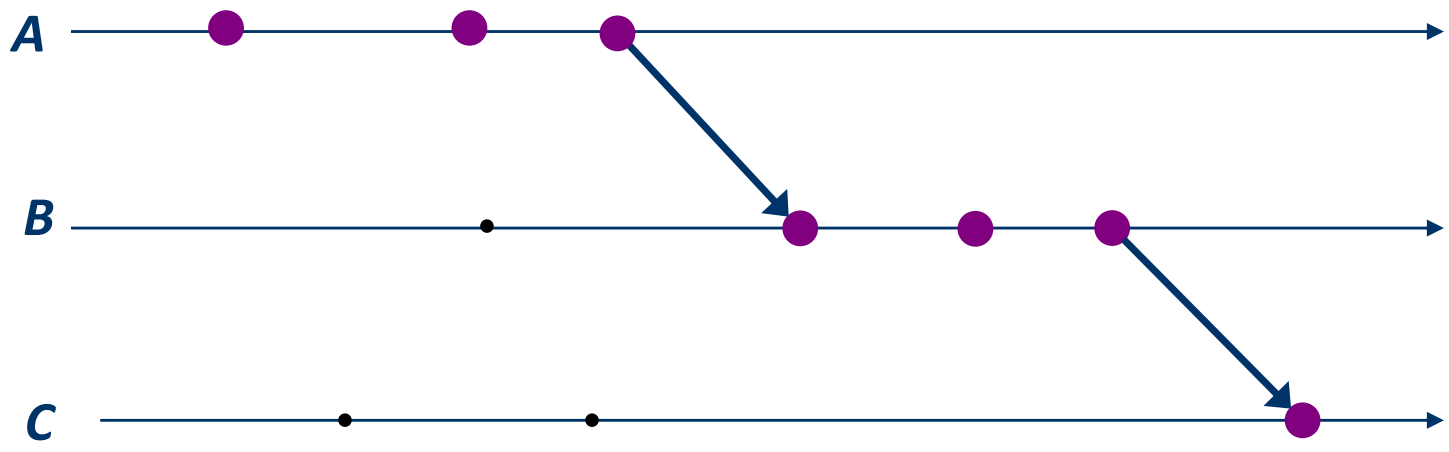


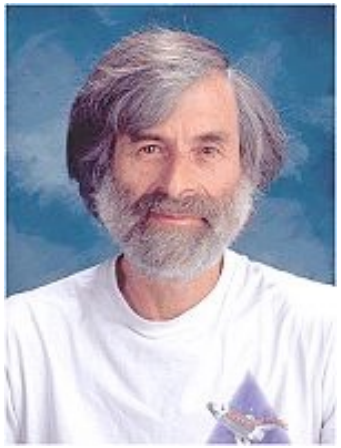


Time, Clocks, and the Ordering of Events in Distributed systems, by Leslie Lamport, CACM 21(7), July 1978

Axiom 3: happened-before (→)

- → is transitive: happened-before is the transitive closure of the relation defined by #1 and #2





Time, Clocks, and the Ordering of Events in Distributed systems, by Leslie Lamport, CACM 21(7), July 1978

Logical time and causality

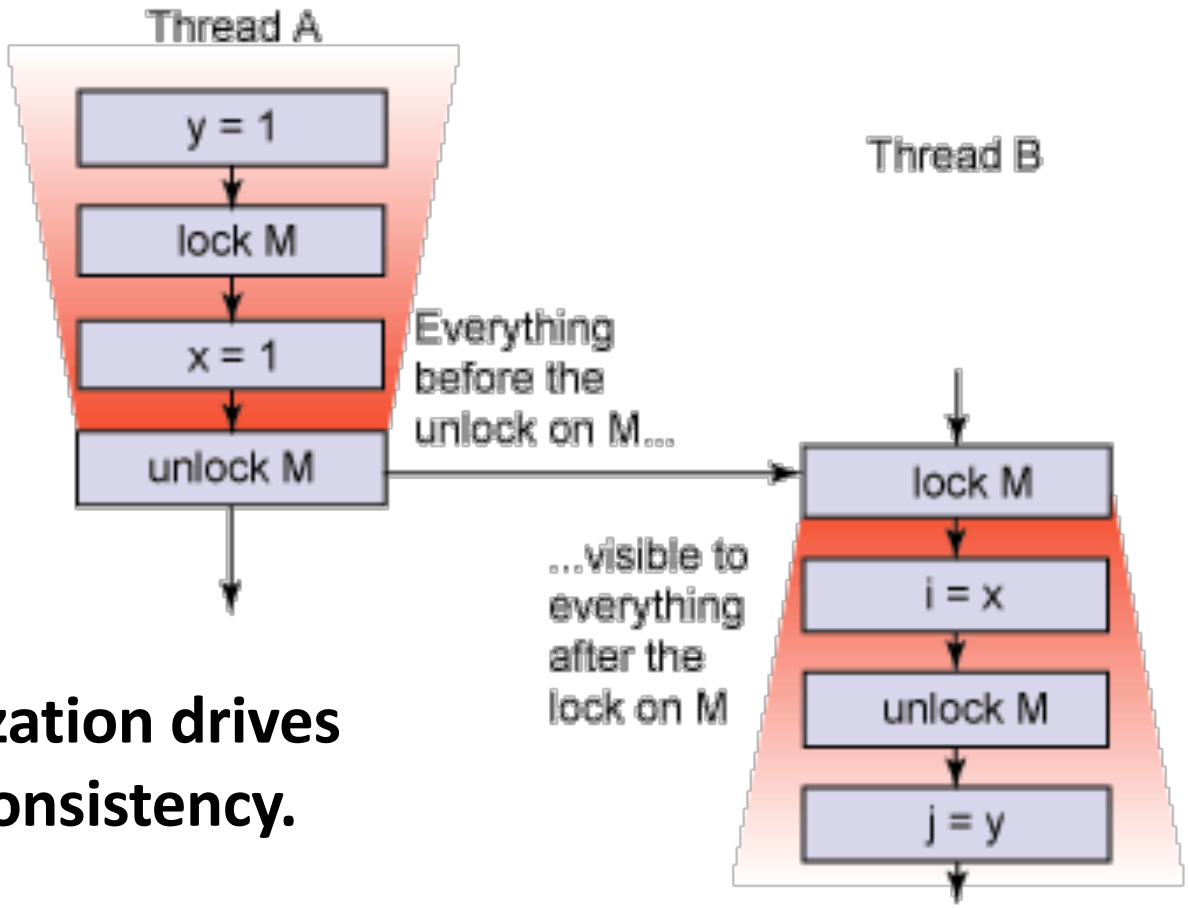
- **Premise**. Nodes communicate only by "touchpoints", e.g., messages: they can observe a remote event only through some chain of messages.
- **Causality**. Event e_1 can affect or cause event e_2 iff the node initiating e_2 could have already observed e_1.
  - Message patterns define a potential causality relation.
  - Also known as happened-before ($\rightarrow$).
- **Logical time**. Event e_1 precedes e_2 iff e_1 could have caused e_2.
  - We say $e_1 \rightarrow e_2$ or $e_1 < e_2$.
- **Concurrency**. Events are concurrent iff neither precedes the other.
  - Happened-before is a **partial order**.

Happened-before is a partial order

- **Partial order**: some events are **concurrent**: no ordering is defined by →.
- Cycles "cannot happen": event e1 cannot occur both before and after e2.
- Therefore, this program cannot print "Hello World".

Thread 1	**Thread 2**
`A = 1`	`B = 1`
`if (B == 0)`	`if (A == 0)`
`print "Hello";`	`print "World";`

Multicore memory systems today



Synchronization drives memory consistency.

Logical clocks

1. Each node maintains a local logical clock value LC.
2. Events/messages/updates are timestamped with the current LC value at the generating node.
   - Increment LC on each new event: LC = LC + 1
   - LC is monotonically increasing
3. Attach the sender's current clock value LC_s to each message ("Piggyback").
4. Receiver advances its LC clock:

if $LC_s > LC$
then $LC = LC_s + 1$

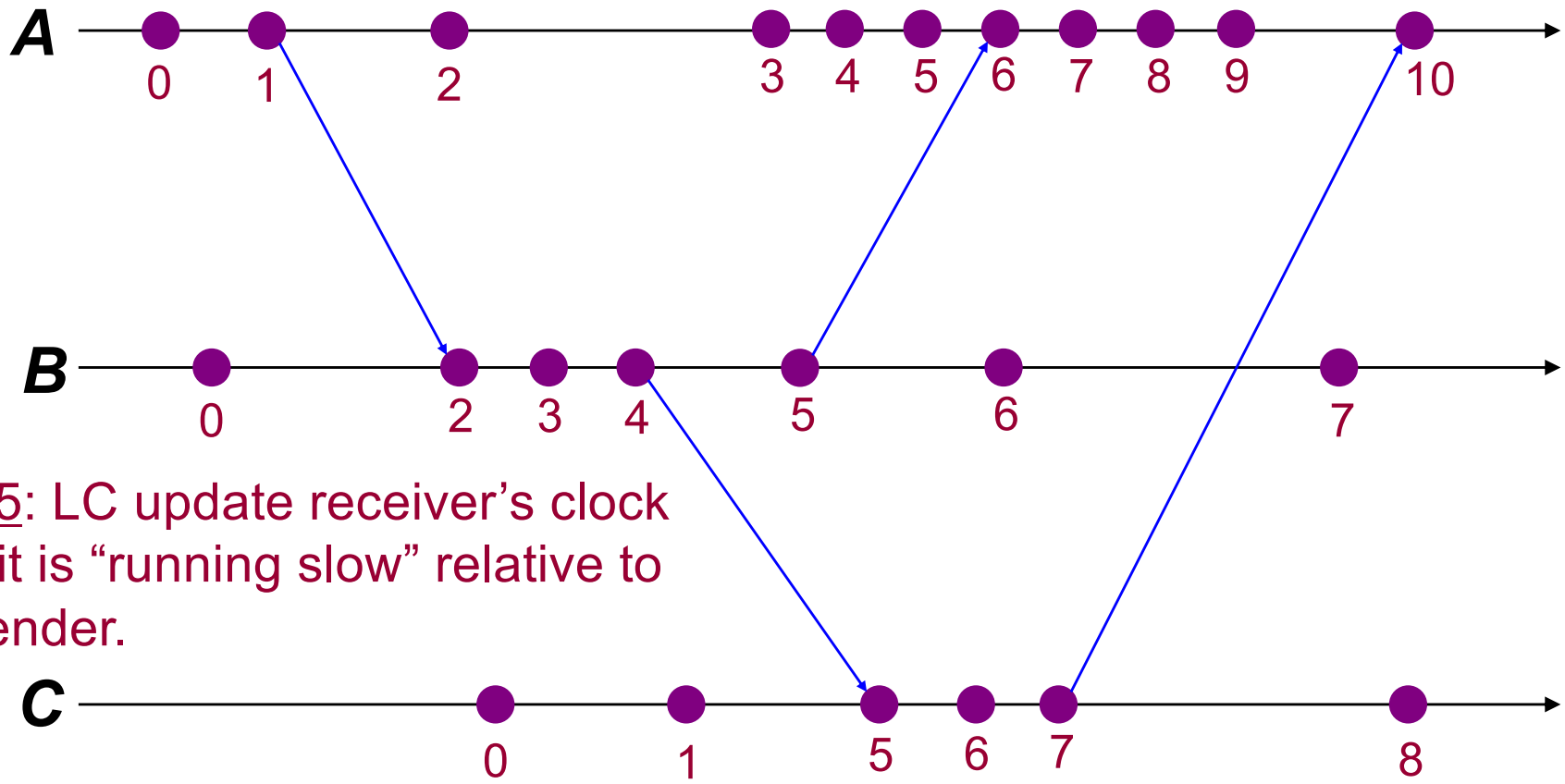
Logical clocks and the clock condition

- *LC* ordering respects potential causality.
- **Clock Condition**: if $e_1 \rightarrow e_2$ then that $LC(e_1) < LC(e_2)$

Causal consistency

- Causal consistency assures that programs see events in a happened-before order: a causal order.
  - Note: there are many such orders.
- We want causal orderings: they capture an essential aspect of what we mean by "consistent".
  - E.g., modern memory systems and languages assure causal orders, at least for correct programs.
- Causal consistency is *weaker* than linearizabiity or sequential consistency.
- Eventual consistency (e.g., Dynamo) does not guarantee causal ordering, resulting in "causal inconsistency".

A6-A10: receiver's clock is unaffected because it is "running fast" relative to sender.



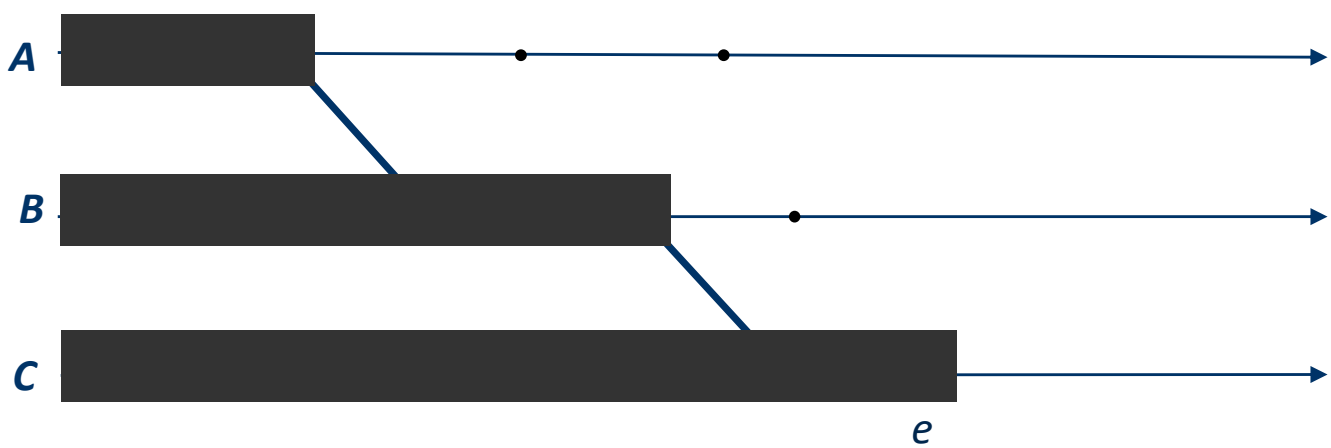
C5: LC update receiver's clock if it is "running slow" relative to sender.

Detecting concurrent events

- How to detect if events are **concurrent**?
- Logical clocks don't help.
- Specifically, the clock condition does not help, because its converse does not hold:
  - *$LC(e_1) < LC(e_2)$ does not imply $e_1 \rightarrow e_2$*
- If *A* observed all events in *B*, then $LC_A > LC_B$
- But just knowing that $LC_A > LC_B$ **does not make it so**.
  - "False positives"
  - "We're gonna need a bigger timestamp!"

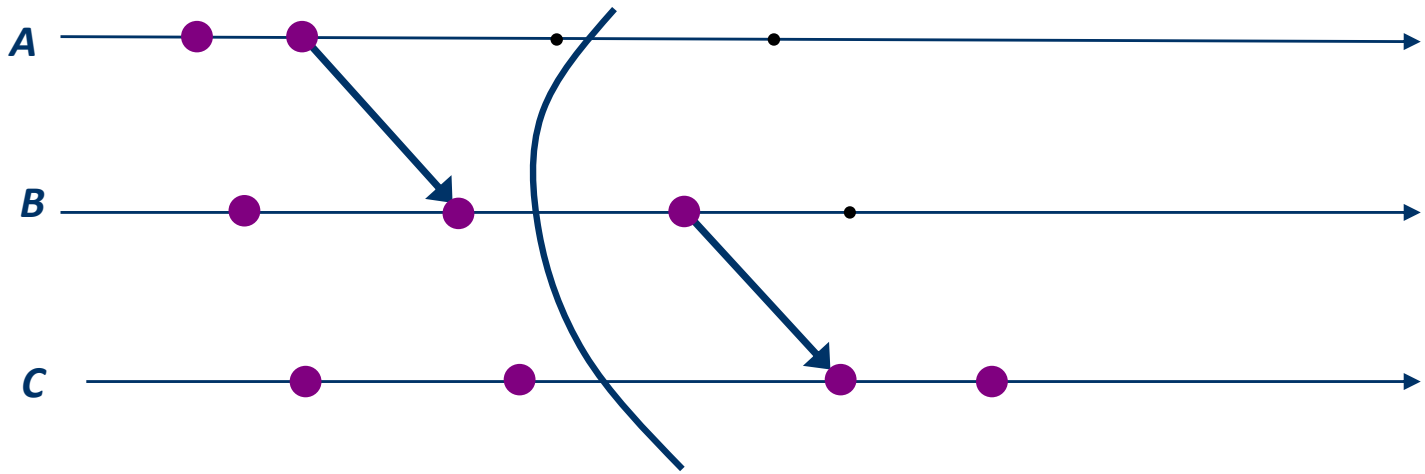
Events in a distributed system

- Sequential history of events in each process/node/thread.
- A local state of a node is defined by a prefix of its **local history**: an event and its predecessors in program order.
- Communication generates causal relationships.
- Shaded prefixes comprise the **causal history** of event *e*.



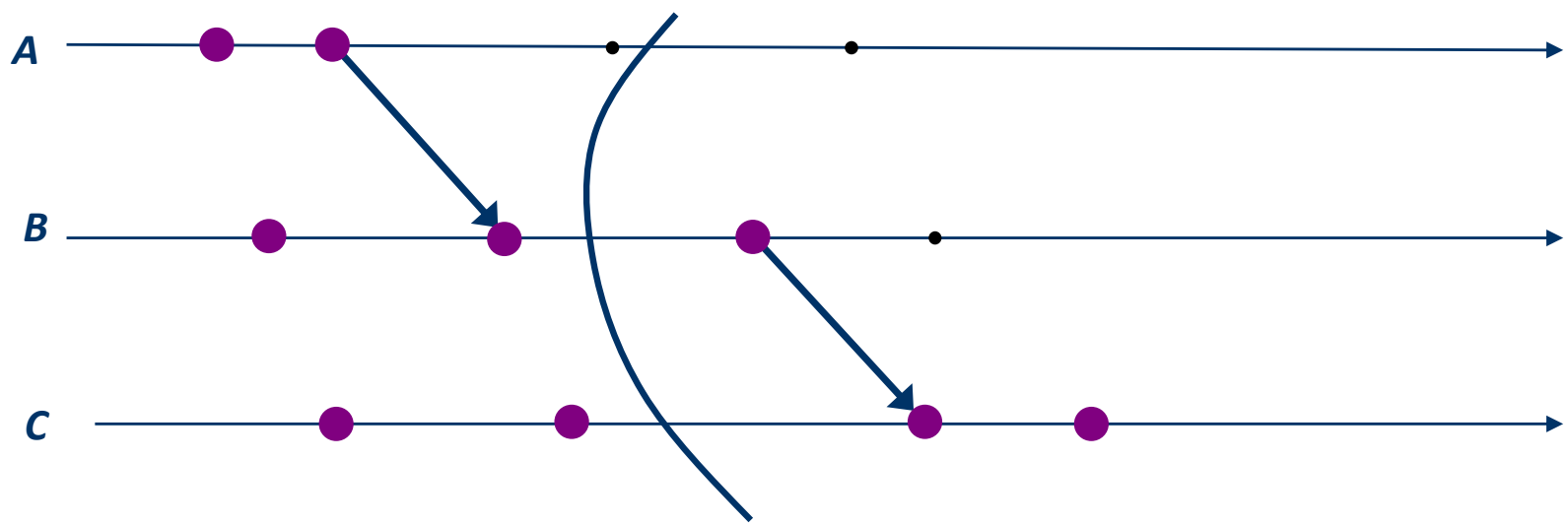
States of a distributed system: cuts

- A cut is a subset of the global history containing a local state (local history prefix) for each node.
- Each cut corresponds to a global state of the system.



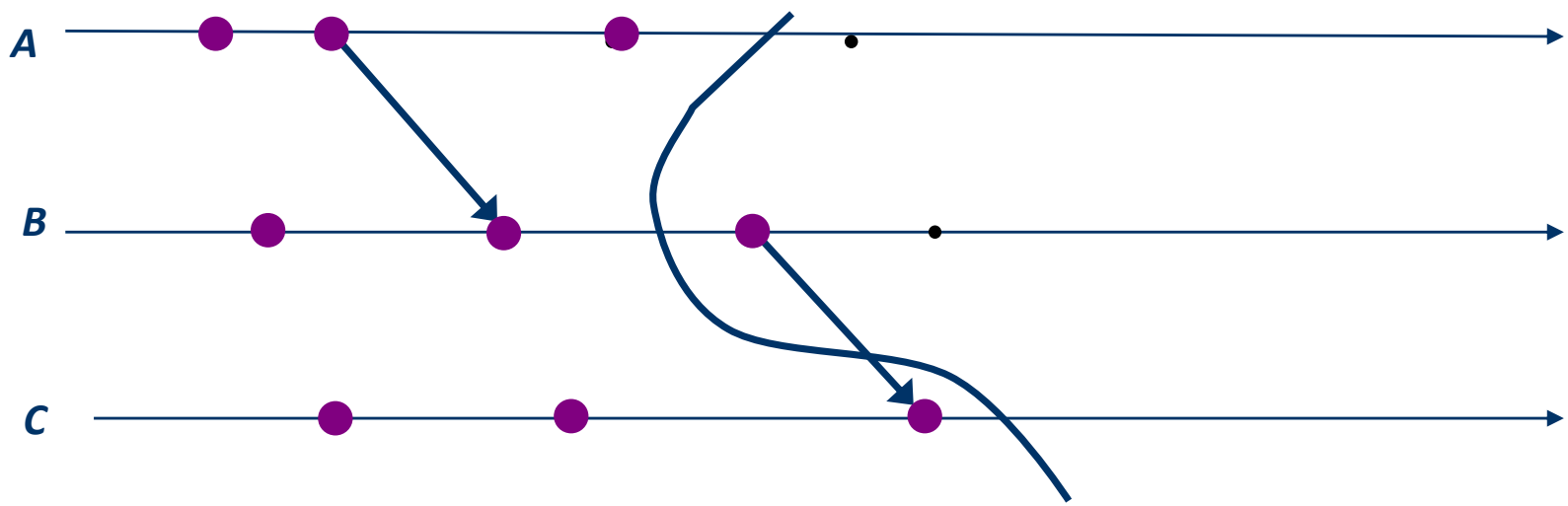
Consistent cuts

- A cut C is a consistent cut iff the following condition is true: for every event $e \in C$, if $\exists e' \rightarrow e$ then $e' \in C$.
- A consistent cut includes the complete causal history of any event in the cut: *left-closed* under *happened-before*.
- C corresponds to a consistent global state.



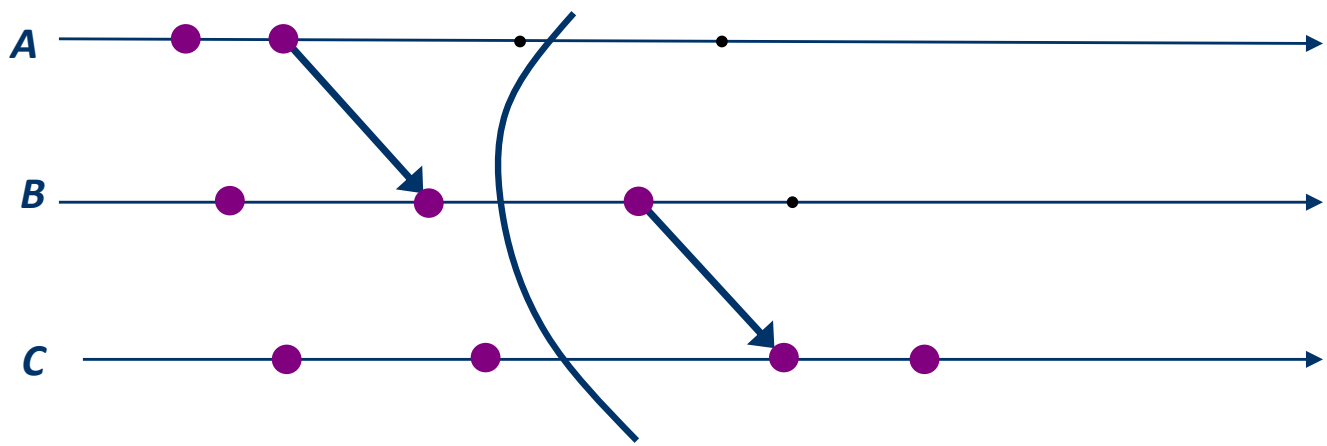
Inconsistency

- An observer of *e* may perceive an inconsistency if it does not know all events in *e*'s causal history.
- A cut C is inconsistent cut if it misses a causal predecessor of some event in the cut: C corresponds to an inconsistent global state.



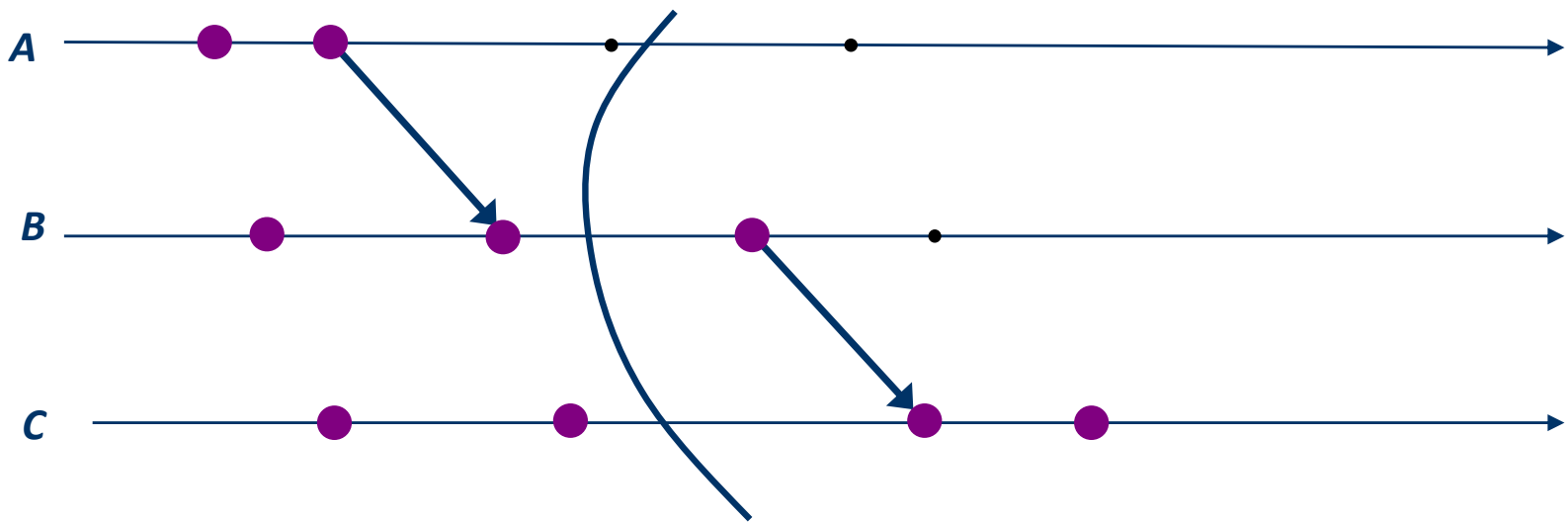
Vector clocks

- **Vector clocks** (also called **vector timestamps**) induce an order that *exactly* reflects causality and concurrency.
- Each of *N* nodes keeps a vector timestamp *TS[N]* as well as a logical clock *LC*.
  - *TS[j]* at node *i* is the most recent value of node *j*'s logical clock that *i* observed or "saw" or "heard about".
  - $TS_i[i] = LC_i$



A vector clock corresponds to a cut

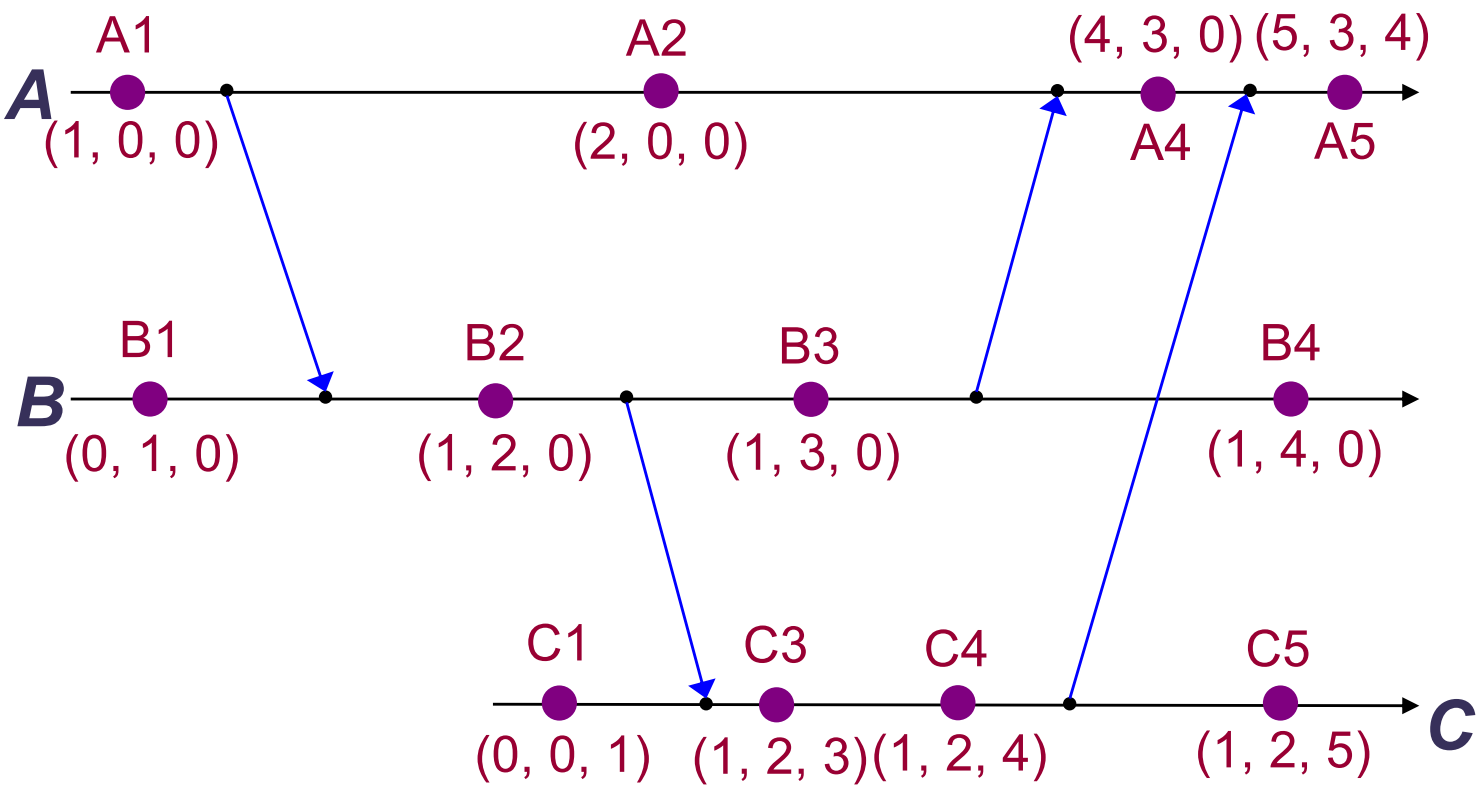
- The **vector timestamp** of a cut is a vector of logical clock values: the local logical clock timestamp of the last event in each local history prefix of the cut.
- Vector timestamps capture observed global states.



Updating a vector clock

- A node *r* observing an event (e.g., receiving a message) from node *s* may learn its causal history.
- Merge (union) sender's causal history with its own.
- Set the TS_r to the pairwise maximum of TS_s and TS_r.
  - $TS_r[i] = \text{MAX}(TS_r[i], TS_s[i])$ for every i

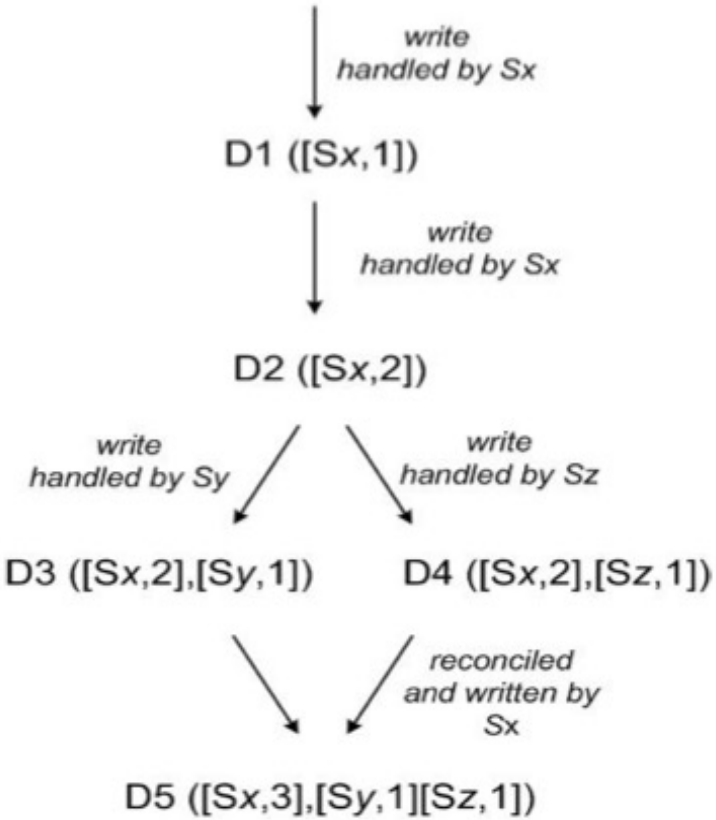
Vector clocks: example



Vector clocks and causality

- *$e_1 \rightarrow e_2$ if and only if $TS(e_2)$ dominates $TS(e_1)$*
  - $e_1 \rightarrow e_2$ iff $TS(e_1)[i] <= TS(e_2)[i]$ for each i
  - "Every event or update visible when e_1 occurred was also visible when e_2 occurred".
- We can determine if two events are **concurrent**: neither *happened-before* the other.
  - If $e_1 \rightarrow e_2$ then $LC(e_1) < LC(e_2)$ and $TS(e_2)$ dominates $TS(e_1)$.
  - If $TS(e_2)$ does not dominate $TS(e_1)$ then it is not true that $e_1 \rightarrow e_2$.
  - If neither vector dominates the other, events are concurrent.

Dynamo timestamps are vector clocks



- A Dynamo vector clock is a list of ([node,count] pairs.
- Each node is a coordinator.
- The count is the number of writes (versions) that the coordinator has handled for the object.

In conclusion

- Causality matters: captures what we mean by consistency.
  - Concurrent updates can be seen in either order.
- Logical clocks capture causality but not concurrency.
- Vector clocks capture causality and concurrency.